Part 4: Class Diagram

4.1 The diagram

Figure 2. Class diagram of Item Borrowing System

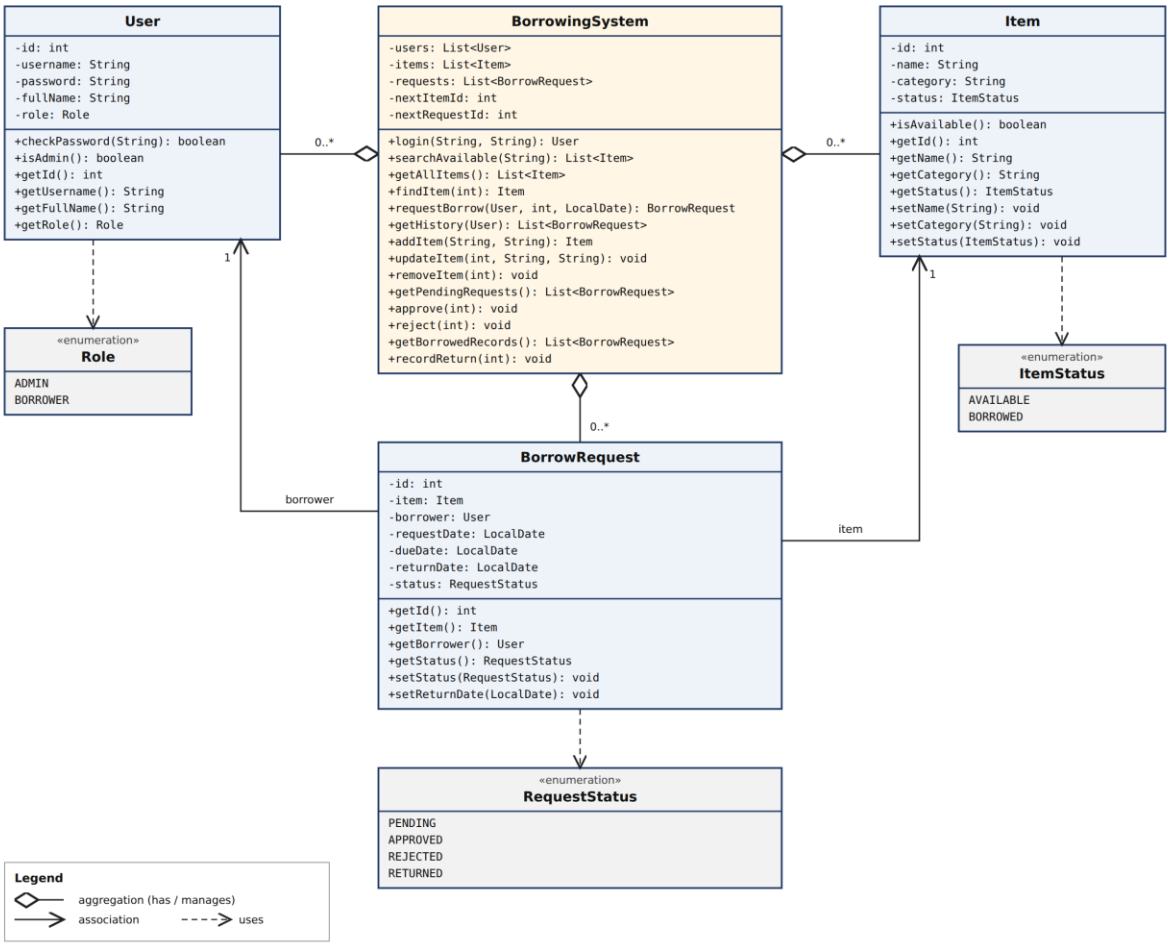


Figure 2. Class diagram of Item Borrowing System

4.2 Class descriptions

Class	What it does and its main members
User	A person who logs in. Holds id, username, password, fullName, and role. checkPassword() verifies a login and isAdmin() tells the system which menu to show.
Item	A piece of equipment that can be borrowed. Holds id, name, category, and status. isAvailable() is used by search and by the borrow rules.
BorrowRequest	One borrowing record. Holds id, the item, the borrower, requestDate, dueDate, returnDate, and status. It is created as "pending" and later becomes approved, rejected, or returned.

Class	What it does and its main members
BorrowingSystem	The controller that keeps the lists of users, items, and requests and applies the rules: login, searchAvailable, requestBorrow, getHistory, addItem, updateItem, removeItem, approve, reject, and recordReturn.
Role	Enumeration with ADMIN and BORROWER. Decides what a user can do.
ItemStatus	Enumeration with AVAILABLE and BORROWED.
RequestStatus	Enumeration with PENDING, APPROVED, REJECTED, and RETURNED.

4.3 Relationships

- **BorrowingSystem** keeps many users, many items, and many borrow requests (aggregation, 0..*).
- **BorrowRequest** refers to exactly one User (the borrower) and exactly one Item (association, 1).
- **User, Item, and BorrowRequest** use the enumerations Role, ItemStatus, and RequestStatus for their fixed sets of values (dependency).

4.4 OOP concepts applied

- **Encapsulation:** every attribute is private and is read or changed only through getters, setters, and methods such as approve() and recordReturn().
- **Enumerations:** roles and statuses are fixed values, so the program cannot store an invalid status.
- **Separation of concerns:** the rules live in BorrowingSystem, so the console menu (Main) and the Swing screens (ItemBorrowingGUI) both reuse the same logic.
- **Exception handling:** invalid actions, such as borrowing an item that is not available, throw IllegalStateException or IllegalArgumentException, and the screens show the message to the user.

The user interface classes (Main and ItemBorrowingGUI) are not drawn in the diagram because they only call BorrowingSystem.

Part 5: Database Design

5.1 The diagram

Figure 3. Entity relationship diagram (database design)

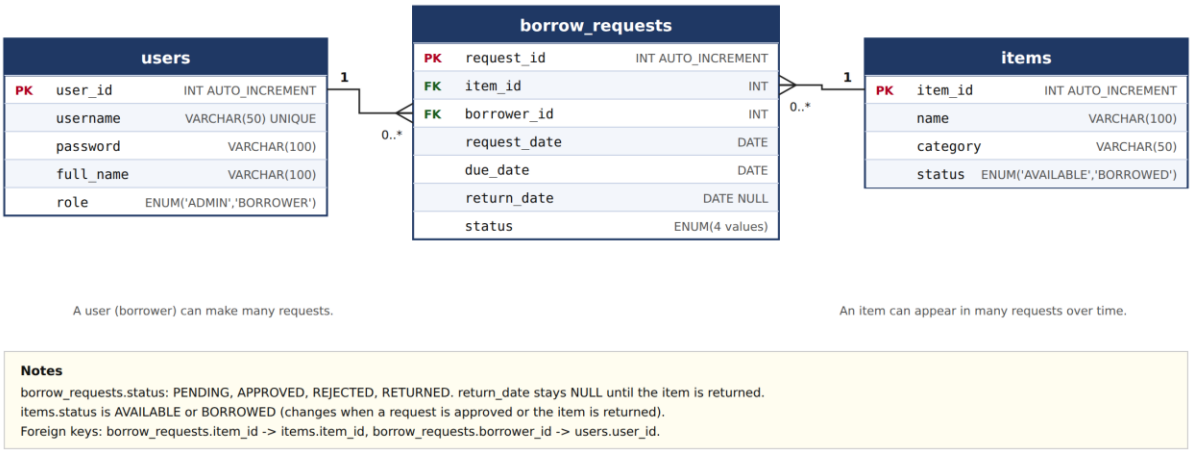


Figure 3. Entity relationship diagram (database design)

5.2 Tables and fields

Table: users (the admin and the borrowers who can log in)

Field	Type	Key	Description
user_id	INT, auto increment	PK	Unique number of the user
username	VARCHAR(50), unique		Name used to log in
password	VARCHAR(100)		Password of the user
full_name	VARCHAR(100)		Name shown on the screens
role	ENUM('ADMIN','BORROWER')		What the user is allowed to do

Table: items (the equipment that can be borrowed)

Field	Type	Key	Description
item_id	INT, auto increment	PK	Unique number of the item
name	VARCHAR(100)		Item name, for example "Epson Projector"
category	VARCHAR(50)		Kind of item, for example "Cable"
status	ENUM('AVAILABLE','BORROWED')		Whether the item can be requested now

Table: borrow_requests (every request, approval, and return)

Field	Type	Key	Description
request_id	INT, auto increment	PK	Unique number of the request
item_id	INT	FK	The item being borrowed (items.item_id)
borrower_id	INT	FK	The borrower who asked (users.user_id)
request_date	DATE		Day the request was sent
due_date	DATE		Return date entered by the borrower
return_date	DATE, can be empty		Day the admin recorded the return; empty until returned
status	ENUM('PENDING','APPROVED','REJECTED','RETURNED')		Stage of the request

5.3 Relationships

- **users to borrow_requests:** one user can make many requests, and each request belongs to one borrower.
- **items to borrow_requests:** one item can appear in many requests over time, and each request is for one item.

5.4 SQL script

The script below creates the database and adds the same sample data used by the program. It is also saved as schema.sql.

```
CREATE DATABASE IF NOT EXISTS item_borrowing;
USE item_borrowing;

CREATE TABLE users (
  user_id    INT AUTO_INCREMENT PRIMARY KEY,
  username   VARCHAR(50)  NOT NULL UNIQUE,
  password   VARCHAR(100) NOT NULL,
  full_name  VARCHAR(100) NOT NULL,
  role       ENUM('ADMIN','BORROWER') NOT NULL
);

CREATE TABLE items (
  item_id    INT AUTO_INCREMENT PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  category   VARCHAR(50)  NOT NULL,
  status     ENUM('AVAILABLE','BORROWED') NOT NULL DEFAULT 'AVAILABLE'
);

CREATE TABLE borrow_requests (
```

```
  request_id    INT AUTO_INCREMENT PRIMARY KEY,
  item_id       INT  NOT NULL,
  borrower_id   INT  NOT NULL,
  request_date  DATE NOT NULL,
  due_date      DATE NOT NULL,
  return_date   DATE NULL,
  status        ENUM('PENDING','APPROVED','REJECTED','RETURNED')
                NOT NULL DEFAULT 'PENDING',
  FOREIGN KEY (item_id)     REFERENCES items(item_id),
  FOREIGN KEY (borrower_id) REFERENCES users(user_id)
);

INSERT INTO users (username, password, full_name, role) VALUES
  ('admin',    'admin123', 'CCS Admin',      'ADMIN'),
  ('student1', 'pass123',  'Juan Dela Cruz', 'BORROWER'),
  ('student2', 'pass123',  'Maria Santos',   'BORROWER');

INSERT INTO items (name, category) VALUES
  ('Epson Projector',   'Projector'),
  ('Extension Cord 5m', 'Electrical'),
  ('HDMI Cable 3m',     'Cable'),
  ('Multimeter',        'Lab tool');
```

Part 6: Screen Design

6.1 Screen list

No.	Screen	Used by	Feature
1	Login	Admin, Borrower	Feature 1: Log in to the system
2	Borrower: Search Items	Borrower	Features 2 and 3: Search for available items, Request to borrow an item
3	Borrower: My Borrowing History	Borrower	Feature 4: View borrowing history
4	Admin: Items	Admin	Features 2, 5, and 6: Search for items, Add a new item, Update or remove an item
5	Admin: Pending Requests	Admin	Feature 7: Approve or reject a borrow request
6	Admin: Borrowed Items	Admin	Feature 8: Record the return of an item
7	Admin: Borrowing History	Admin	Feature 4: View borrowing history
8	Request to Borrow dialog	Borrower	Feature 3: Request to borrow an item
9	Add or Update Item dialog	Admin	Features 5 and 6: Add a new item, Update an item

6.2 Screen sketches

Each sketch shows the layout of one screen. The sample data matches the data in the database script.

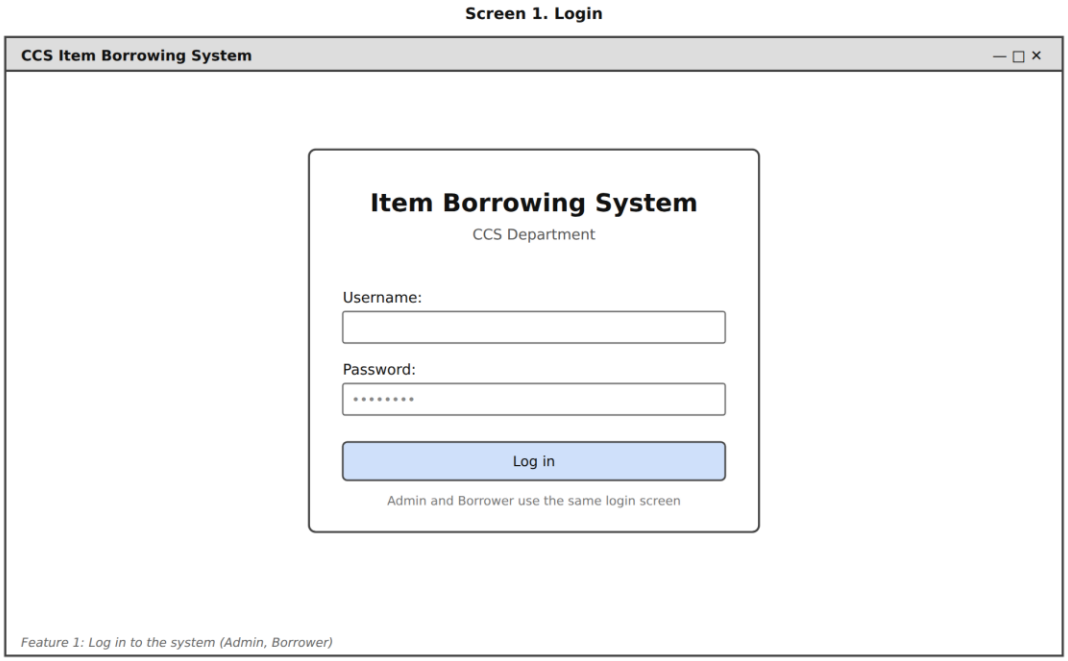


Figure 4. Screen 1: Login

Both users enter a username and password. The system opens the admin screens or the borrower screens depending on the role.

Screen 2. Borrower – Search Items

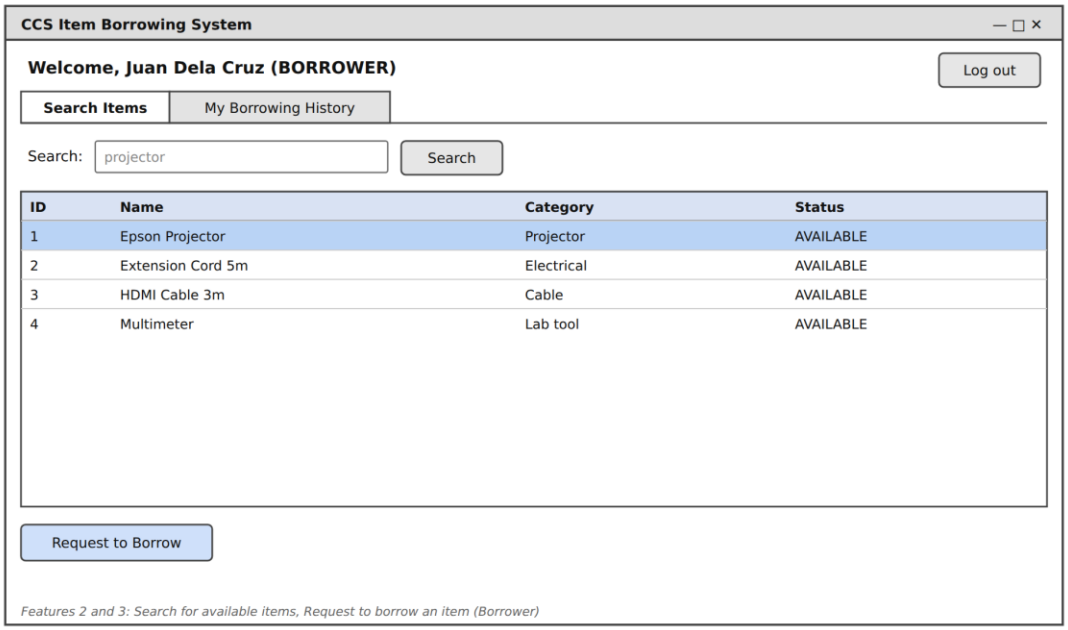


Figure 5. Screen 2: Borrower: Search Items

The borrower types a keyword, sees only available items, selects one, and clicks "Request to Borrow".

Screen 3. Borrower – My Borrowing History

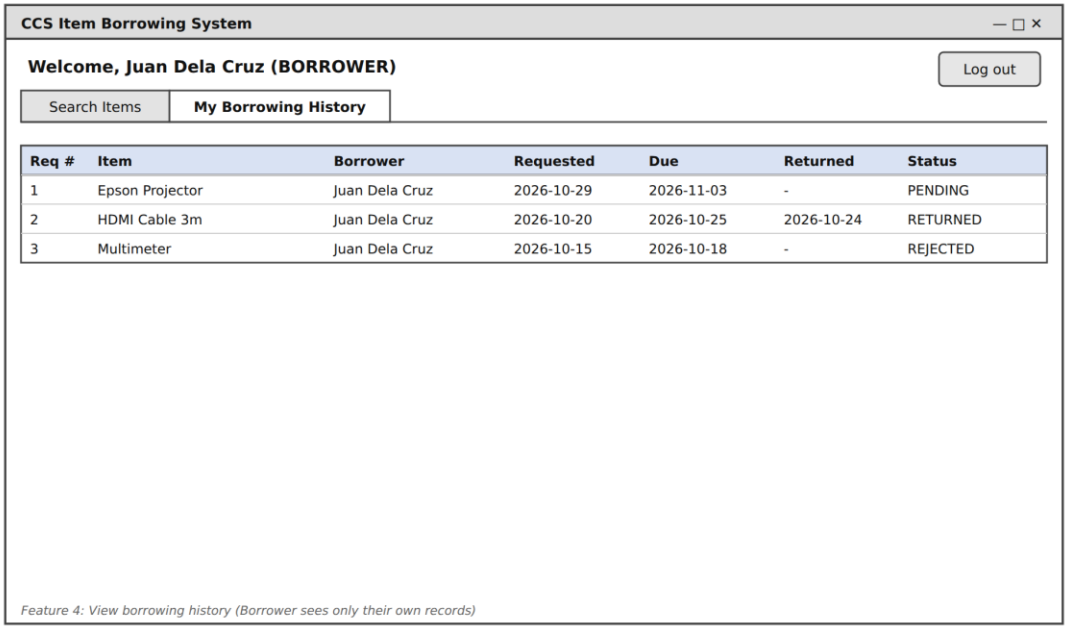


Figure 6. Screen 3: Borrower: My Borrowing History

A table of the borrower's own requests with their due dates, return dates, and status.

Screen 4. Admin – Items

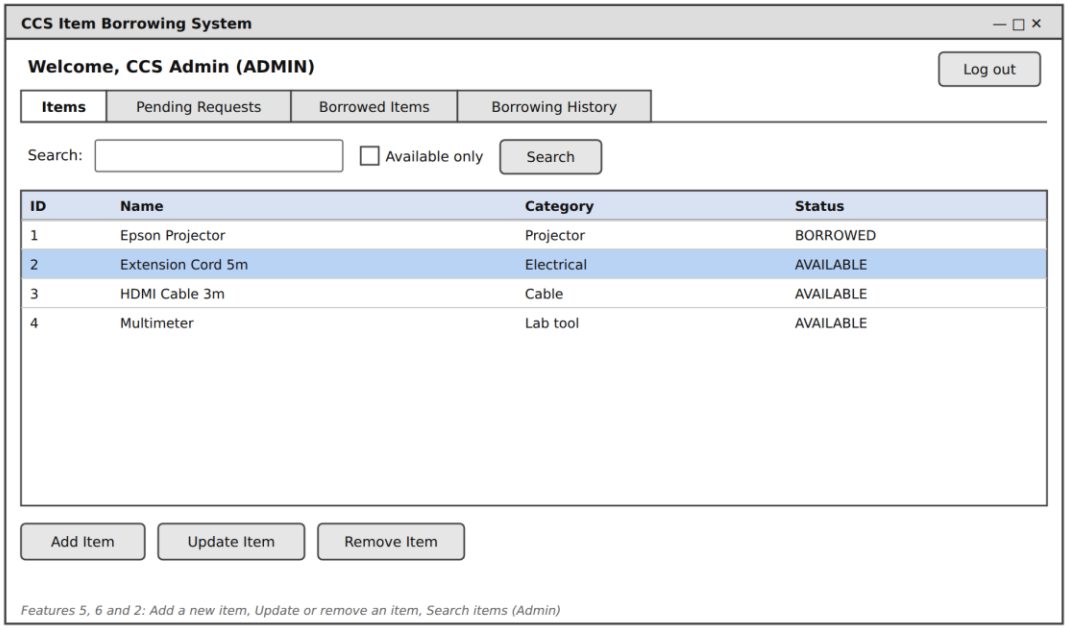


Figure 7. Screen 4: Admin: Items

The admin sees every item, can filter to available ones, and uses the buttons to add, update, or remove an item.

Screen 5. Admin – Pending Requests

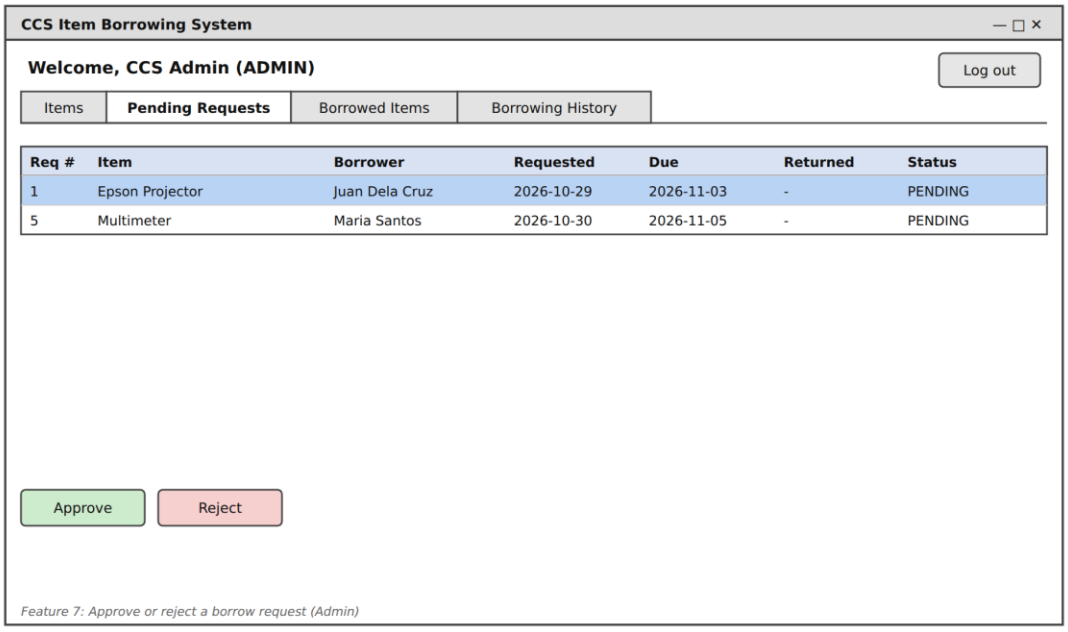


Figure 8. Screen 5: Admin: Pending Requests

The admin selects a pending request and clicks Approve or Reject.

Screen 6. Admin – Borrowed Items

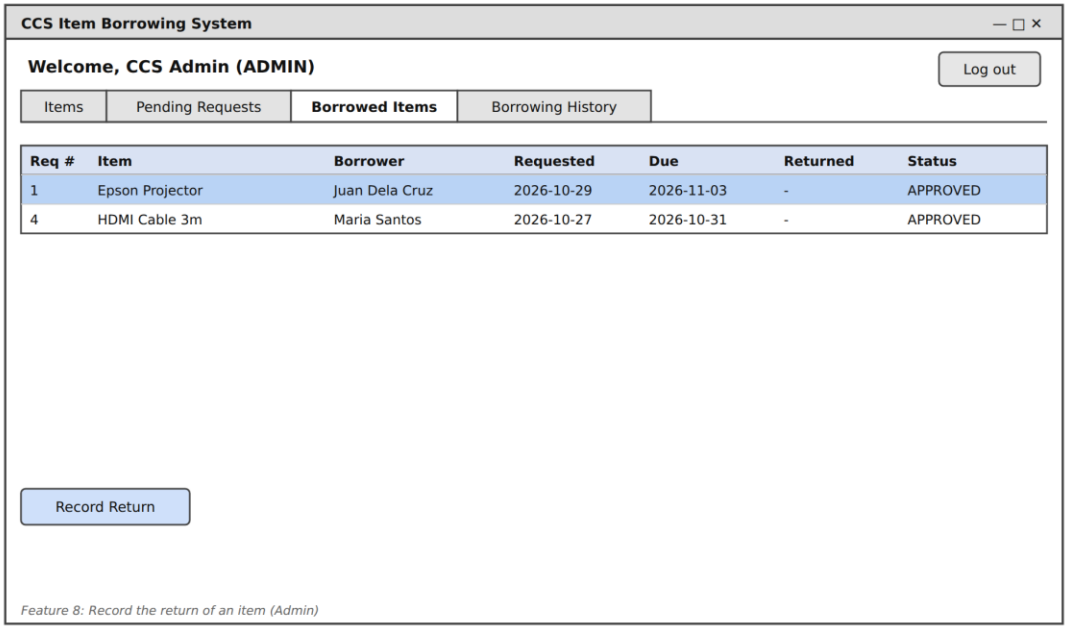


Figure 9. Screen 6: Admin: Borrowed Items

The admin selects the borrow record of the item that was brought back and clicks "Record Return".

Screen 7. Admin – Borrowing History

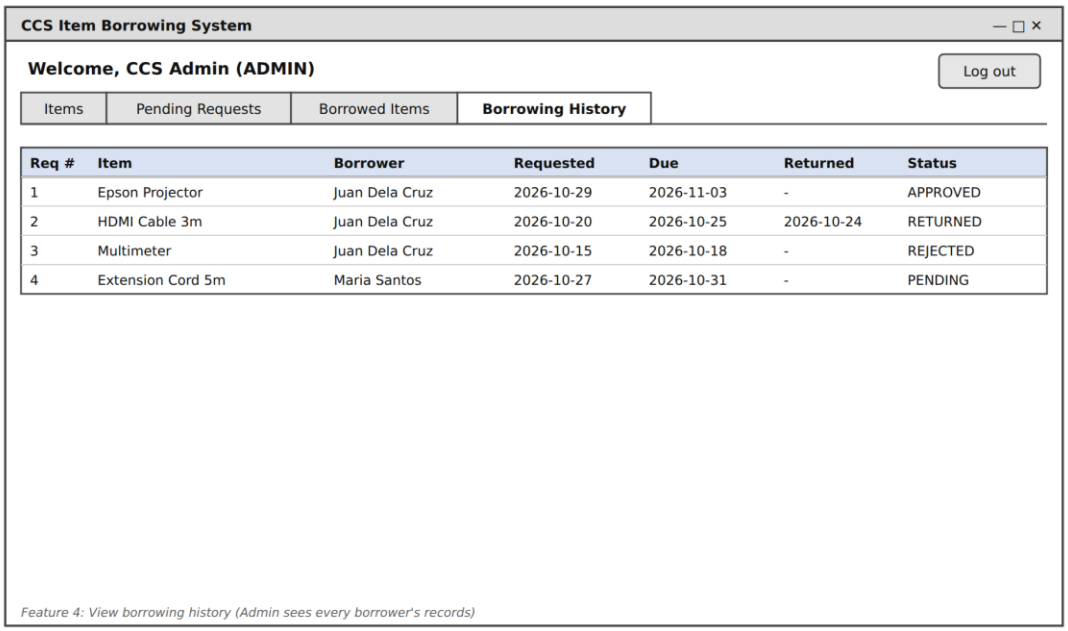


Figure 10. Screen 7: Admin: Borrowing History

A table of every borrower’s requests, so the admin can see who borrowed each item and when it was returned.

Screen 8. Request to Borrow dialog

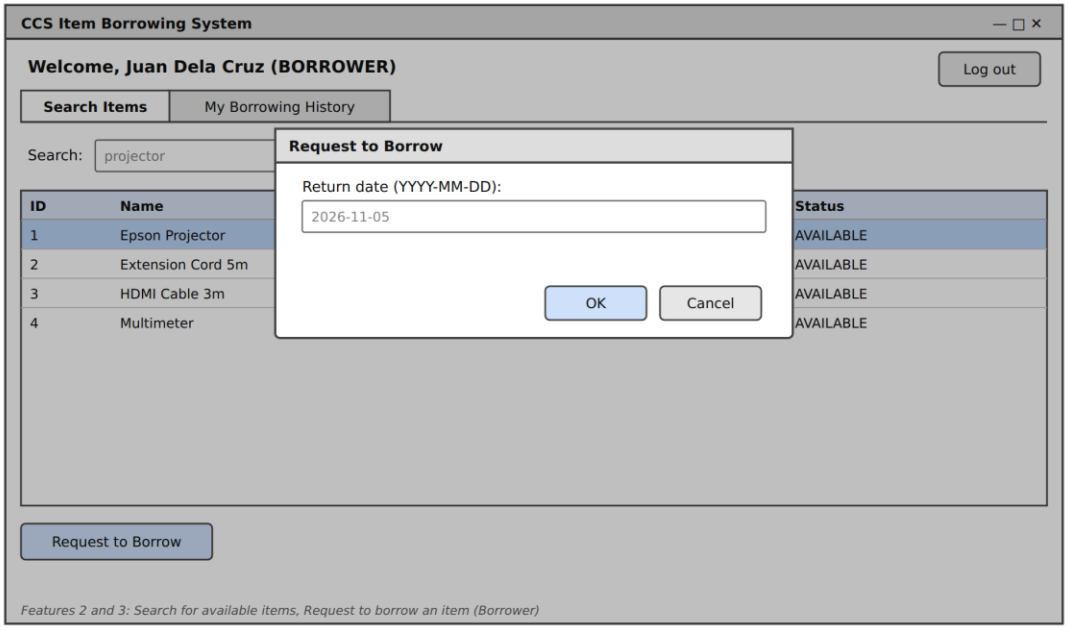


Figure 11. Screen 8: Request to Borrow dialog

A small window that asks for the date the item will be returned. The default is seven days from today.

Screen 9. Add / Update Item dialog

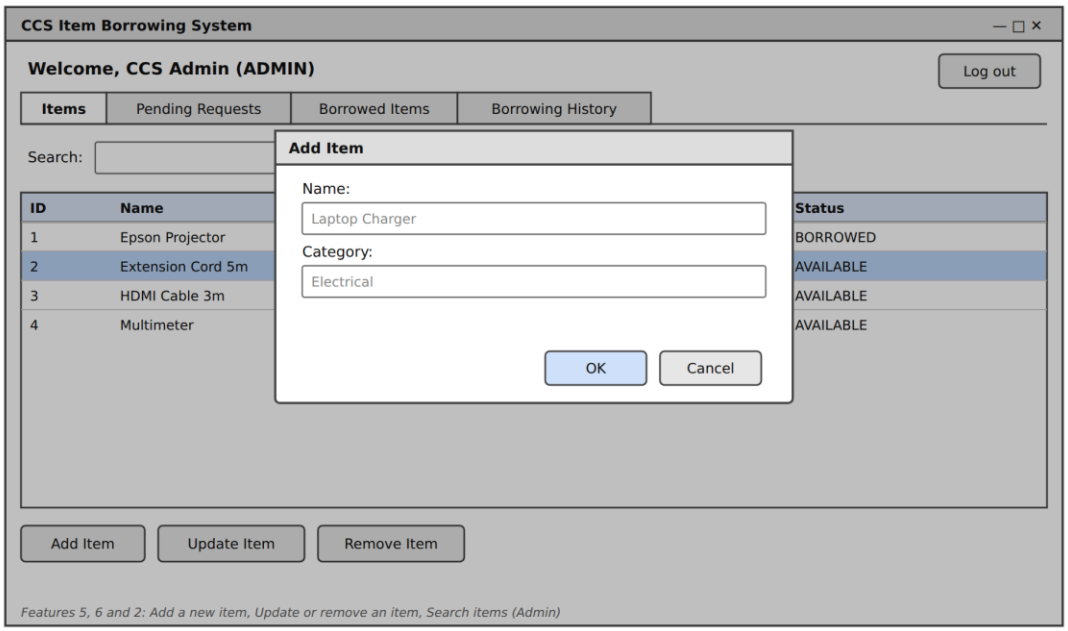


Figure 12. Screen 9: Add or Update Item dialog

A small window with the item name and category. When updating, the fields start with the current values.

6.3 Screenshots of the real screens

The real screens are made with Java Swing in ItemBorrowingGUI. Run the program, then place one screenshot under each label below.

- *[Insert screenshot of Screen 1: Login]*
- *[Insert screenshot of Screen 2: Borrower: Search Items]*
- *[Insert screenshot of Screen 3: Borrower: My Borrowing History]*
- *[Insert screenshot of Screen 4: Admin: Items]*
- *[Insert screenshot of Screen 5: Admin: Pending Requests]*
- *[Insert screenshot of Screen 6: Admin: Borrowed Items]*
- *[Insert screenshot of Screen 7: Admin: Borrowing History]*
- *[Insert screenshot of Screen 8: Request to Borrow dialog]*
- *[Insert screenshot of Screen 9: Add or Update Item dialog]*

Part 7: Work Plan

7.1 Team roles

Member	Main role	Responsibilities in the work plan
Hadrian Esparcia	Documenter	Keeps the project documentation up to date, builds the login and borrower screens, and writes the user guide and reflection.
Deanver C. Gudin	Diagram lead	Draws and updates the class diagram, database diagram, and screen sketches, and builds the admin screens.
John Patrick H. Escaran	Reviewer	Builds the core classes and the database connection, checks every part before it is submitted, and leads the testing.

7.2 Schedule

Task	Owner	Start	End	Status
Parts 1 to 3: overview, users, use case diagram	All members	Oct 26	Oct 29	Done
Part 4: class diagram	Deanver	Nov 2	Nov 4	Draft ready
Part 5: database design and SQL script	John	Nov 2	Nov 5	Draft ready
Part 6: screen sketches	Deanver	Nov 2	Nov 6	Draft ready
Part 7: work plan	Hadrian	Nov 2	Nov 3	Draft ready
Core classes and rules (BorrowingSystem)	John	Nov 3	Nov 9	Draft ready
Login and borrower screens	Hadrian	Nov 9	Nov 16	Planned
Admin screens	Deanver	Nov 9	Nov 16	Planned
Database connection (MySQL)	John	Nov 10	Nov 18	Planned
Joining the parts and fixing bugs	All members	Nov 17	Nov 22	Planned
Part 8: testing	John	Nov 20	Nov 25	Planned
Part 9: user guide, source code link, reflection	Hadrian	Nov 23	Nov 29	Planned
Final review and submission	All members	Nov 30	Dec 1	Planned

The dates are a plan and can be moved to match the class deadline.

7.3 Checkpoints

- Every part is reviewed by John before it is added to the final document.

- The group meets after each major step (diagrams, screens, database, testing) to check that the parts work together.
- Any task that falls behind is shared with the other members so the work can be re-assigned.